

 Serviço Nacional de Aprendizagem Industrial Santa Catarina	AVALIAÇÃO PRÁTICA		Desempenho
	Data: 26/08/2025		
	Docente: Marcio Schoenfelder		
	Curso Técnico em Desenvolvimento de Sistemas		
	Unidade Curricular: Programa o Orientada a Objetos		
	Turma: QA IDM 2025/1 V1		
	Estudante: David Luis Kim		

CAPACIDADES
Conteúdo da Situação de Aprendizagem - Cadastro e Consulta com Dart + MySQL

CONTEXTUALIZAÇÃO
Você desenvolver Dart, faz parte de uma equipe responsável por criar um protótipo simples para registro e consulta de dados de clientes e pedidos utilizando Dart conectado diretamente ao MySQL, sem uso de API intermediária. O sistema deve permitir incluir clientes e pedidos, além de realizar duas consultas distintas: (1) listagem dos pedidos com dados do cliente; (2) resumo de pedidos por cliente com total gasto.

DESAFIO
Implementar duas classes relacionadas (Cliente e Pedido) com persistência de dados em banco MySQL. Criar as tabelas necessárias, implementar inclusão (INSERT) e seleção (SELECT) com dois tipos de consulta distintos, exibindo os resultados no console.

RESULTADOS E ENTREGAS
Detalhamento das Atividades e Resultados Esperados • Modelagem das classes Cliente e Pedido: Classes modeladas com atributos, métodos e relação 1:N definida. Exposição dialogada + prática guiada; • Criação do schema e tabelas no MySQL: Script SQL criado, testado e funcionando. Prática em laboratório; • Implementação da conexão em Dart e funções de INSERT: Inclusões funcionando para Cliente e Pedido. Demonstração + exercício individual; • Implementação dos SELECTs (JOIN e GROUP BY): Consultas retornam dados corretos e formatados. Exercício dirigido; • Organização do código e entrega final: Projeto funcional, organizado e documentado. Checklist de entrega. Lista de Verificação (Avaliação) Classes Cliente e Pedido implementadas com relação correta: A modelagem está adequada ao problema proposto?

Script SQL funcional com chaves primárias e estrangeira: As tabelas e relações foram criadas corretamente no MySQL?

Funções de INSERT implementadas: As inserções executam sem erros e salvam os registros corretamente?

Dois SELECTs distintos implementados: Existe um SELECT com JOIN e outro com GROUP BY funcionando corretamente?

Saída formatada no console: As informações são exibidas de forma clara e organizada?

ORIENTAÇÕES ADICIONAIS

Recursos Necessários

- Laboratório de Informática
- Projetor Multimídia
- Acesso à Internet
- Dart SDK e VS Code
- MySQL Server / MariaDB
- Biblioteca mysql_client para Dart

PRINTS DE COMPROVAÇÃO

Print 1: Tela MySQL, Workbench verificando todos usuários.

The screenshot displays the MySQL Workbench interface. The 'Query' window contains the following SQL query:

```
SELECT * FROM mysql.user;
```

The 'Result Grid' shows the output of the query, displaying columns: Host, User, Select_priv, Insert_priv, Update_priv, Delete_priv, Create_priv, Drop_priv, Reload_priv, Shutdown_priv, Process_priv, and File_priv. The results are as follows:

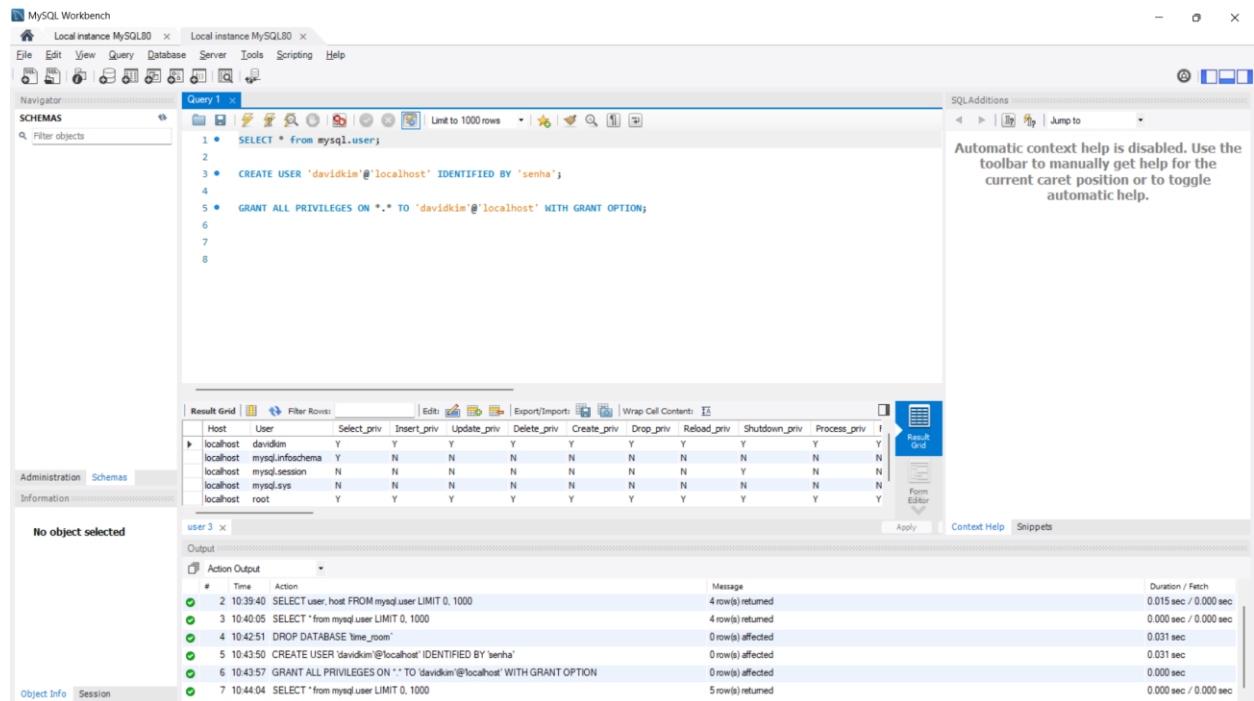
Host	User	Select_priv	Insert_priv	Update_priv	Delete_priv	Create_priv	Drop_priv	Reload_priv	Shutdown_priv	Process_priv	File_priv
localhost	mysql.infoschema	Y	N	N	N	N	N	N	N	N	N
localhost	mysql.session	N	N	N	N	N	N	N	Y	N	N
localhost	mysql.sys	N	N	N	N	N	N	N	N	N	N
localhost	root	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y

The 'Output' window at the bottom shows the execution log with the following messages:

```
# Time Action Message Duration / Fetch
1 10:38:56 select * from sq_user LIMIT 0, 1000 Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCH... 0.000 sec
2 10:39:40 SELECT user, host FROM mysql.user LIMIT 0, 1000 4 row(s) returned 0.015 sec / 0.000 sec
3 10:40:05 SELECT * from mysql.user LIMIT 0, 1000 4 row(s) returned 0.000 sec / 0.000 sec
```

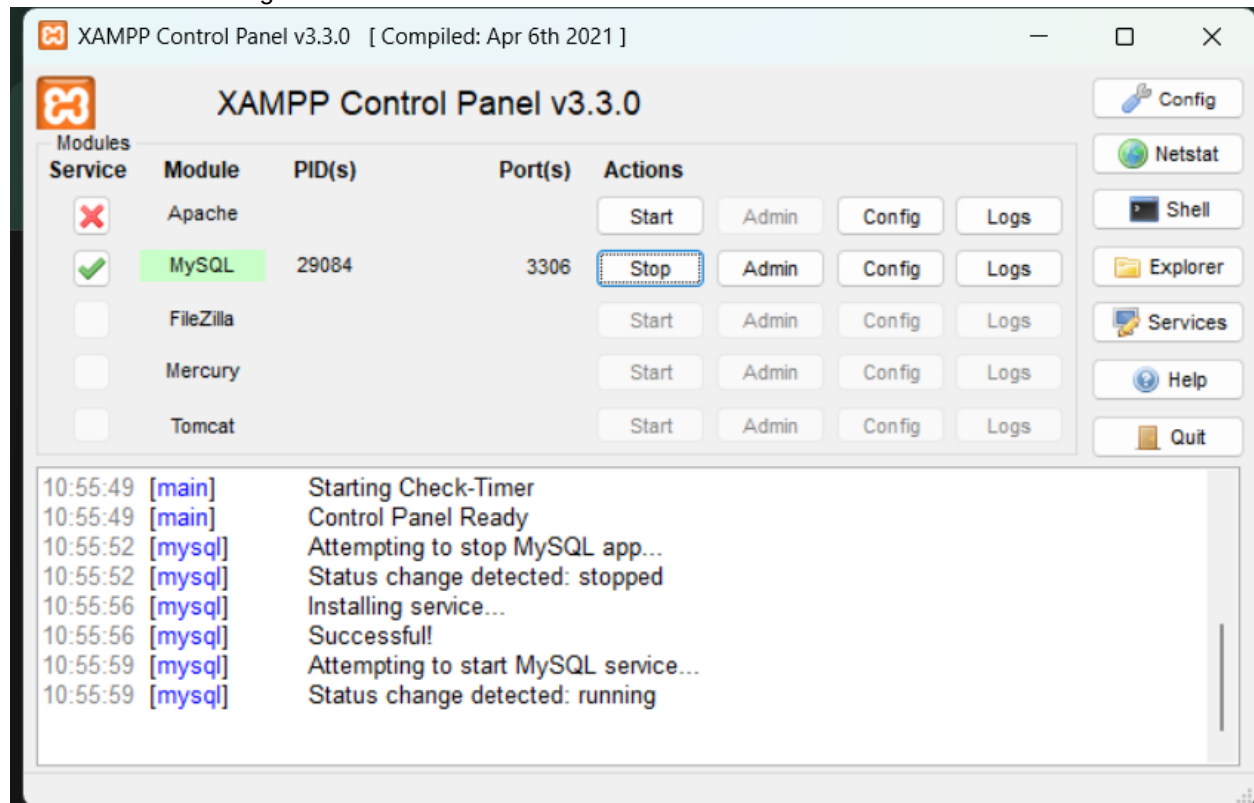
Após instalado xampp, verifiquei os usuários que continham em meu MySQL Workbench e decidi criar um usuário chamado davidkim.

Print 2: Tela MySQL Workbench, após permissionamento do usuário davidkim.



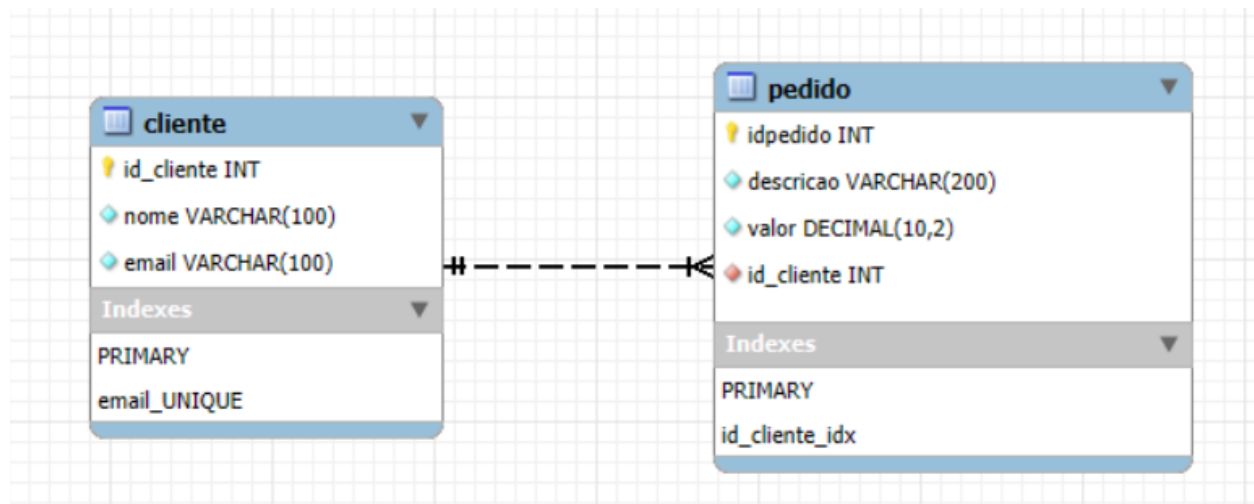
Após criado usuário davidkim garanti todas as permissões para ele. E rodei o xampp.

Print 3: XAMPP configurado e rodando.



Após XAMPP configurado, criamos as tabelas no Mysql Workbench, fazendo o diagrama e os comandos para criação da tabela.

Print 4: Relacionamento das tabelas pedido e cliente.

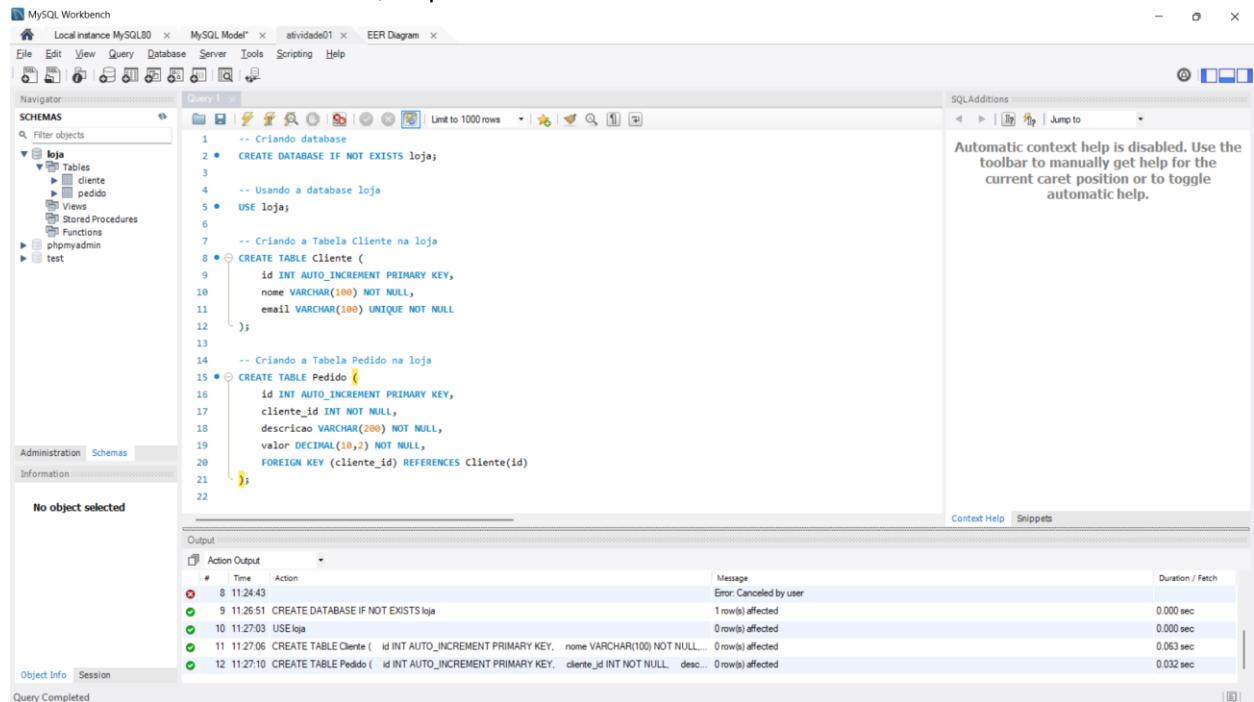


Print 5: Comando SQL utilizado para criar database loja e tabelas.

```
1  -- Criando database
2  • CREATE DATABASE IF NOT EXISTS loja;
3
4  -- Usando a database loja
5  • USE loja;
6
7  -- Criando a Tabela Cliente na loja
8  • CREATE TABLE Cliente (
9      id INT AUTO_INCREMENT PRIMARY KEY,
10     nome VARCHAR(100) NOT NULL,
11     email VARCHAR(100) UNIQUE NOT NULL
12 );
13
14 -- Criando a Tabela Pedido na loja
15 • CREATE TABLE Pedido (
16     id INT AUTO_INCREMENT PRIMARY KEY,
17     cliente_id INT NOT NULL,
18     descricao VARCHAR(200) NOT NULL,
19     valor DECIMAL(10,2) NOT NULL,
20     FOREIGN KEY (cliente_id) REFERENCES Cliente(id)
21 );
```

Após isso obtivemos o resultado das tabelas e database criado.

Print 6: comandos executados, esquema criado e tabelas existindo.



Código main.dart:



main.txt

Primeiro a necessidade com a conexão do banco de dados, então instalei as dependências no pubspec.yaml do mysql workbench, rodei dart pub get, e após inserir o package: mysql1, comecei os códigos.

Print 7: classes representadas por códigos no .dart

```

/// -----
/// Models
/// -----

/// Representa um cliente da loja
class Cliente {
  int? id;          // ID do cliente (autogerado pelo MySQL)
  String nome;      // Nome do cliente
  String email;     // Email do cliente (único)

  /// Construtor da classe Cliente
  Cliente({this.id, required this.nome, required this.email});
}

/// Representa um pedido de um cliente
class Pedido {
  int? id;          // ID do pedido (autogerado pelo MySQL)
  int clienteId;    // ID do cliente que fez o pedido (chave estrangeira)
  String descricao; // Descrição do pedido
  double valor;     // Valor do pedido

  /// Construtor da classe Pedido
  Pedido({
    this.id,
    required this.clienteId,
    required this.descricao,
    required this.valor,
  });
}

```

Print 8: Código funcionando no main.dart para conectar com banco de dados MySQL.

```

/// -----
/// Conexão com MySQL
/// -----

/// Função para criar e retornar uma conexão com o banco de dados MySQL
Future<MySQLConnection> getConnection() async {
  final settings = ConnectionSettings(
    host: 'localhost',    // Endereço do servidor MySQL
    port: 3306,           // Porta padrão do MySQL
    user: 'davidkim',     // Usuário do banco
    password: 'senha',    // Senha do usuário
    db: 'loja',           // Nome do banco de dados
  );
  return await MySQLConnection.connect(settings);
}

```

Print 9: Funções para inserir

```
/// -----  
/// Funções de INSERT  
/// -----  
  
/// Insere um novo cliente no banco de dados  
Future<void> inserirCliente(Cliente cliente) async {  
    final conn = await getConnection();  
    await conn.query(  
        'INSERT INTO Cliente (nome, email) VALUES (?, ?)',  
        [cliente.nome, cliente.email],  
    );  
    await conn.close();  
}  
  
/// Insere um novo pedido no banco de dados  
Future<void> inserirPedido(Pedido pedido) async {  
    final conn = await getConnection();  
    await conn.query(  
        'INSERT INTO Pedido (cliente_id, descricao, valor) VALUES (?, ?, ?)',  
        [pedido.clienteId, pedido.descricao, pedido.valor],  
    );  
    await conn.close();  
}
```

Print 10: função de selecionar

```
/// -----  
/// Funções de SELECT  
/// -----  
  
/// Lista todos os pedidos com os dados do cliente correspondente  
Future<void> listarPedidosComClientes() async {  
  final conn = await getConnection();  
  var results = await conn.query('''  
    SELECT Pedido.id, Pedido.descricao, Pedido.valor, Cliente.nome, Cliente.email  
    FROM Pedido  
    INNER JOIN Cliente ON Pedido.cliente_id = Cliente.id  
  ''');  
  
  // Exibe cada pedido no console  
  for (var row in results) {  
    print(  
      'Pedido #${row[0]} - ${row[1]} | R${row[2]} - Cliente: ${row[3]} (${row[4]}',  
    );  
  }  
  await conn.close();  
}  
  
/// Mostra um resumo do total gasto por cada cliente  
Future<void> resumoPedidosPorCliente() async {  
  final conn = await getConnection();  
  var results = await conn.query('''  
    SELECT Cliente.nome, SUM(Pedido.valor) AS total_gasto  
    FROM Cliente  
    INNER JOIN Pedido ON Cliente.id = Pedido.cliente_id  
    GROUP BY Cliente.nome  
  ''');  
  
  // Exibe o total gasto por cliente no console  
  for (var row in results) {  
    print('Cliente: ${row[0]} - Total Gasto: R${row[1]}');  
  }  
  await conn.close();  
}
```

Print 11: void main

```
/// Função principal do programa
/// Executa inserções e consultas no banco de dados
Run | Debug
void main() async {
    // -----
    // Inserção de clientes exemplo
    // -----
    /*

    await inserirCliente(Cliente(nome: 'João Silva', email: 'joao@email.com'));
    await inserirCliente(Cliente(nome: 'Maria Souza', email: 'maria@email.com'));

    */

    // -----
    // Inserção de pedidos exemplo
    // -----

    /*
    await inserirPedido(
        Pedido(clienteId: 1, descricao: 'Notebook', valor: 3500.00),
    );
    await inserirPedido(
        Pedido(clienteId: 1, descricao: 'Mouse', valor: 150.00),
    );
    await inserirPedido(
        Pedido(clienteId: 2, descricao: 'Celular', valor: 2200.00),
    );
    */

    // -----
    // Consultas e exibição no console
    // -----
    print('\n--- Listagem de Pedidos com Clientes ---');
    await listarPedidosComClientes();

    print('\n--- Resumo de Pedidos por Cliente ---');
    await resumoPedidosPorCliente();
}
```

após rodar o comando `dart run` no `main.dart`:

Print 12: lista de produtos printados no terminal.

```
--- Listagem de Pedidos com Clientes ---  
Pedido #1 - Notebook | R$3500.0 - Cliente: João Silva (joao@email.com)  
Pedido #2 - Mouse | R$150.0 - Cliente: João Silva (joao@email.com)  
Pedido #3 - Celular | R$2200.0 - Cliente: Maria Souza (maria@email.com)  
  
--- Resumo de Pedidos por Cliente ---  
Cliente: João Silva - Total Gasto: R$3650.0  
Cliente: Maria Souza - Total Gasto: R$2200.0
```

e para checar as tabelas no mysql workbench utilizamos comandos como `select`:

Print 13: `Select` para ver pedido, descrição valor, e o cliente que comprou com id, nome e email.

```
23 • SELECT  
24     Pedido.id AS pedido_id,  
25     Pedido.descricao,  
26     Pedido.valor,  
27     Cliente.id AS cliente_id,  
28     Cliente.nome,  
29     Cliente.email  
30 FROM Pedido  
31 INNER JOIN Cliente ON Pedido.cliente_id = Cliente.id;  
32
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content:

	pedido_id	descricao	valor	cliente_id	nome	email
▶	1	Notebook	3500.00	1	João Silva	joao@email.com
	2	Mouse	150.00	1	João Silva	joao@email.com
	3	Celular	2200.00	2	Maria Souza	maria@email.com

Print 14: Select para ver o cliente, com id, nome, email e pedido com valores.

```
33 • SELECT
34     Cliente.id AS cliente_id,
35     Cliente.nome,
36     Cliente.email,
37     Pedido.id AS pedido_id,
38     Pedido.descricao,
39     Pedido.valor
40 FROM Cliente
41 LEFT JOIN Pedido ON Cliente.id = Pedido.cliente_id;
42
43
44
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	cliente_id	nome	email	pedido_id	descricao	valor
▶	1	João Silva	joao@email.com	1	Notebook	3500.00
	1	João Silva	joao@email.com	2	Mouse	150.00
	2	Maria Souza	maria@email.com	3	Celular	2200.00

Print 14: select para ver clientes e totais gastos:

```
43 • SELECT
44     c.id AS cliente_id,
45     c.nome,
46     c.email,
47     IFNULL(SUM(p.valor), 0) AS total_gasto
48 FROM Cliente c
49 LEFT JOIN Pedido p ON c.id = p.cliente_id
50 GROUP BY c.id, c.nome, c.email;
51
52
53
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
cliente_id	nome	email	total_gasto
1	João Silva	joao@email.com	3650.00
2	Maria Souza	maria@email.com	2200.00